

East African Journal of Information Technology
eajit.eanso.org
Volume 9, Issue 2, 2026
Print ISSN: 2707-5346 | Online ISSN: 2707-5354
Title DOI: https://doi.org/10.37284/2707-5354

EAST AFRICAN NATURE & SCIENCE ORGANIZATION

Original Article

The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models

Naveen Kumar Vedurupaka[1*]

[1] Independent Researcher, USA.
[*] Author for Correspondence Email; naveenkumarvedurupakausa@gmail.com

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

Date Published:

07 October 2026

Keywords:

Natural language understanding, Conversational AI, Large language models, Transformer architecture, Model serving infrastructure, Intent classification, Slot filling, Dialogue state tracking, Inference optimisation, Distributed systems.

ABSTRACT

Conversational Natural Language Understanding (NLU) has passed through five distinguishable technological eras: rule-based symbolic parsing, statistical and probabilistic modelling, neural sequence learning, Transformer-based pretraining, and the current large language model regime. Existing accounts of this history tend to foreground modelling advances and treat infrastructure as a supporting detail. This article argues instead that the two tracks are causally entangled: each scientific advance in conversational NLU became production-viable only once a matching infrastructure capability, spanning hardware, data pipelines, training paradigms, and serving architecture, made it deployable at scale, and each infrastructure capability in turn exposed the next scientific bottleneck. The analysis traces a specific bottleneck sequence, from combinatorial rule brittleness through representational discreteness, vanishing gradients, and sequential processing limits, to labelled-data scarcity, showing how each resolution reshaped the engineering problems that remained. A closing synthesis argues that large language models have not simplified conversational architecture; they have relocated its complexity from authoring rules and training classifiers toward orchestration, retrieval grounding, and failure isolation across distributed inference systems. The account draws on systems-architecture reasoning applied to the published research and technical-report record, rather than on a new empirical benchmark.

APA CITATION

Vedurupaka, N. K. (2026). The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models. *East African Journal of Information Technology*, *9*(2), 456-472. https://doi.org/10.37284/eajit.9.2.5954.

CHICAGO CITATION

Vedurupaka, Naveen Kumar. (2026). "The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models". *East African Journal of Information Technology,* 9 (2), 456-472. https://doi.org/10.37284/eajit.9.2.5954.

HARVARD CITATION

Vedurupaka, N. K. (2026). "The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models", *East African Journal of Information Technology*, 9(2), pp. 456-472. doi: 10.37284/eajit.9.2.5954.

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

IEEE CITATION

N. K. Vedurupaka, "The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models", *EAJIT*, vol. 9, no. 2, pp. 456-472, Oct. 2026.

MLA CITATION

Vedurupaka, Naveen Kumar. (2026). "The Model-Infrastructure Co-Evolution in Conversational NLU: From ELIZA to Large Language Models", *EAJIT*, vol. 9, no. 2, pp. 456-472, Oct. 2026, doi:10.37284/eajit.9.2.5954.

INTRODUCTION

Natural Language Understanding (NLU) in conversational systems transforms a user utterance, conditioned on dialogue history, into a machine-usable representation capable of driving a system's next action: executing a transaction, retrieving an answer, requesting clarification, or generating a response. This definition excludes machine translation, document summarisation, and standalone linguistic analysis tasks such as dependency parsing, except where those disciplines contributed techniques later absorbed into conversational systems. Conversational NLU operates under constraints- turn-taking, dialogue state, real-time interaction- that general text processing does not share, and these constraints shape the engineering decisions at the centre of this article.

The central argument is that advances in conversational NLU have rarely been purely scientific breakthroughs. Each major transition, from rule-based parsers to statistical models to neural architectures to Transformer-based pretraining to large language models, was enabled by an equally significant change in infrastructure: the hardware, data pipelines, training paradigms, and operational patterns that made a given model practically deployable at scale. Each new infrastructure capability then exposed application requirements that motivated the next scientific advance. This reciprocal dynamic, rather than a smooth accumulation of modelling improvements, appears to be the organising principle behind conversational NLU's six-decade trajectory [1].

Read this way, the history looks like a chain of bottlenecks in which resolving one problem exposes the next. Combinatorial brittleness in hand-written rules gave way to representational discreteness in early statistical models; representational discreteness gave way to vanishing gradients in early recurrent networks; vanishing gradients gave way to sequential processing constraints once gated recurrent architectures matured; and sequential processing constraints gave way to labelled-data scarcity once self-attention removed the need for step-by-step computation. Pretraining and few-shot learning addressed data scarcity, at the cost of a new set of reliability and governance problems that now define the large language model era. This framing suggests why the field's history reads less like a smooth curve of improvement and more like a sequence of paradigm shifts, each triggered when the prior constraint was resolved, and the next became visible.

Five eras structure the account that follows: rule-based symbolic NLU (1960s to 1980s), where hand-crafted grammars and expert systems were limited by brittleness and single-machine infrastructure; statistical and probabilistic NLU (1990s to 2000s), where hidden Markov models, maximum-entropy classifiers, and conditional random fields learned from annotated corpora rather than hand-written rules; neural sequence models (2013 to 2016), where recurrent networks, attention, and GPU acceleration enabled context-aware multi-turn dialogue; Transformer-based pretraining and transfer learning (2017 to 2020), where self-attention and large-scale pretraining reduced the annotation burden for new domains; and the large language model era (2020 to the present), where instruction-tuned, few-shot capable models running on global inference fleets are reshaping conversational system design.

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

Conversational systems make this dynamic unusually visible because they cannot defer their computation the way document classification or offline translation can. A conversational system must respond within a bounded turn-time budget, maintain state across an open-ended number of turns, and adapt to feedback the user supplies in real time. Latency budgets shape model-size decisions; dialogue-state persistence shapes database and session-store architecture; human feedback loops shape data-pipeline design. Enterprise teams choosing between a single-model and a pipeline serving topology, between sub-200-millisecond and multi-second latency budgets, or between fixed-capacity classifiers and per-token inference pricing are, in effect, choosing among assumptions inherited from a specific era's engineering paradigm, whether or not that inheritance is made explicit.

Each era is examined across four linked pillars: scientific innovation (the modelling paradigm and its failure modes), engineering architecture (training, serving, and scaling infrastructure), unlocked applications (product capabilities that became feasible), and industry impact (how the surrounding software landscape changed). These pillars are treated as causally connected rather than independent: accuracy gains unlock applications, applications drive scale, scale motivates infrastructure investment, and infrastructure investment enables the next round of scientific experimentation.

The contribution offered here is a causal engineering synthesis rather than a chronological survey or a benchmark comparison. Section 2 sets out the recurring NLU subtasks used as a scorecard across eras. Sections 3 through 7 trace the five historical regimes in sequence. Section 8 synthesises cross-cutting patterns and argues that the large language model era has relocated architectural complexity rather than removed it. Section 9 concludes with implications for practitioners designing the next generation of conversational platforms.

MAIN BODY OF THE PAPER

The NLU Problem Space as a Cross-Era Scorecard

Production conversational NLU systems repeatedly solve a small set of subtasks, and using them as a fixed scorecard makes it possible to compare eras on common ground rather than on shifting benchmarks alone.

Intent classification maps an utterance to one or more predefined action categories and is typically scored by precision, recall, and F1, with confidence thresholds enabling fallback when the classifier is uncertain. Architecturally, it is the lightest subtask in the stack: stateless, single-pass, and tolerant of aggressive caching, which is why production challenges here tend to concern confidence calibration rather than raw accuracy. Slot filling identifies and types the parameters within an utterance and depends on the active intent, since the same word or span can play different roles under different intents. Dialogue state tracking (DST) maintains a running record of confirmed slots, pending clarifications, and task completion across turns, and imposes the heaviest architectural demands of the core subtasks: session-persistent storage, state representations that grow with conversation depth, and recovery paths for mid-conversation failures. Context carryover and reference resolution, resolving "it" or "make it Chicago instead" to the correct prior entity, is tightly coupled to DST and is the subtask where rule-based and statistical systems failed most visibly [2].

Semantic parsing, which maps utterances directly to executable representations such as SQL queries or API calls, became reliable only with neural sequence-to-sequence models and is now central to large language model tool invocation. Negation and modality detection, sentiment and emotion detection, and semantic similarity became

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

production-relevant only once representations moved beyond hand-built heuristics, since each depends on capturing meaning that surface-level pattern matching cannot express.

These subtasks cluster into three operational regimes with distinct architectural signatures. Stateless single-turn tasks, intent classification and domain routing among them, are cacheable and impose sub-10-millisecond latency budgets; their challenges concern confidence calibration and out-of-scope detection rather than throughput. Stateful multi-turn tasks, DST and reference resolution, require session-persistent storage and consistency guarantees, and their failures, stale state or missed coreference, are subtle rather than catastrophic but compound across turns. Generative and executable tasks, semantic parsing, retrieval-augmented generation, and tool invocation, produce outputs with side effects and demand sandboxed execution, retrieval-index freshness, and rollback paths that earlier NLU subtasks did not require. Each era addressed a different subset of these regimes, and the boundary between what was architecturally feasible and what remained out of reach shifted with every paradigm.

These subtasks are also operationally measurable in ways that make cross-era comparison tractable: intent classification and domain routing by accuracy and macro-F1, slot filling by span-level precision, recall, and F1, dialogue state tracking by joint goal accuracy on multi-domain benchmarks, and semantic parsing by exact-match or execution accuracy. Confidence calibration and thresholding are not incidental to this scorecard; they are the mechanism by which a probabilistic or generative component communicates its own uncertainty to a downstream policy layer, and every era examined below implements some version of this contract even where the underlying model class changes entirely.

Era I: Rule-Based Symbolic NLU (1960s–1980s)

The earliest conversational systems applied mechanical pattern matching to transform utterances into responses. ELIZA matched user input against hand-written regular expressions and substitution rules to generate Rogerian-therapist-style replies, demonstrating both the illusion of understanding that surface-level matching can produce and the limits of that illusion, since the program held no model of syntax, semantics, or dialogue context [3]. Later systems pursued more structured representations: SHRDLU parsed utterances into logical forms, checked semantic consistency, and executed actions in a simulated blocks world, while expert-system shells such as MYCIN chained symbolic rules for medical diagnosis, reaching specialist-comparable accuracy within a narrow domain [4,5].

These systems performed well within narrow domains but proved brittle under ordinary linguistic variation. A rule written for "book me a flight to New York" would fail on "I'd like to travel to NYC," because coverage depended on vocabulary variation, syntactic variation, ellipsis and anaphora, and negation scope, each of which required separate hand-authored treatment. The response was to engineer around brittleness through more elaborate grammars, dialogue state machines, and frame-and-slot structures that defined task-level templates with named parameters, narrowing the NLU problem to selecting the correct frame and extracting its slot values. For narrow, transaction-oriented domains this was workable; for anything broader, each new rule interacted with every existing rule through shared vocabulary and dialogue-state side effects, producing a maintenance cost that grew faster than the rule count. By the 1980s, government funding reviews had drawn attention to these limits, contributing to a reduction in AI research investment [6]. The DARPA-funded ATIS programme, which built spoken interfaces to airline databases, would later prove important less for its accuracy than for the annotated corpus it generated,

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

since that corpus became the standard testbed bridging the rule-based and statistical eras [2].

Telephony automation was the largest commercial outlet for this technology, though it is worth distinguishing two tiers within it. The dominant deployment was touch-tone menu navigation, which involved no language processing at all, only branching on keypad input. A smaller tier layered rule-based speech recognition and keyword spotting atop the same call-flow infrastructure, attempting to handle spoken commands such as "check my balance" through pattern matching. Both tiers shared the finite-state call-flow architecture described below, but only the latter constituted NLU in any meaningful sense; the touch-tone tier is relevant here mainly because it established the call-routing metrics and uptime expectations that rule-based NLU systems inherited once they entered the same telephony stack.

The runtime architecture mirrored this simplicity. A single process matched the utterance against an ordered rule base, selected the first or highest-priority match, populated a template's slots, and issued a canned response or a clarification prompt if nothing matched. There was no retraining pipeline, no online learning, and no data-driven feedback path; updates meant editing rules and redeploying. Dialogue state reduced to a position in a finite-state graph plus a partially filled slot frame, typically a few hundred bytes, which made the system trivially serialisable: rollback meant resetting to a prior node, and certification meant enumerating reachable states exhaustively. For regulated deployments such as banking interactive voice response (IVR) and medical triage scripts, this deterministic auditability was a prerequisite rather than a convenience, a property that no later NLU paradigm would fully preserve. ELIZA itself ran on an IBM 7094 with roughly 144 KB of core memory under MIT's timesharing system, a hardware footprint consistent with a single-process, latency-tolerant, offline design [3].

Despite this brittleness, rule-based systems found commercial footing in narrow, high-value domains, DTMF-tree IVR menus, help-desk FAQ systems, and structured technical-support walkthroughs, precisely because domain narrowness kept the combinatorial explosion manageable. Measured against the subtask scorecard of Section 2, the era made intent classification feasible only within rigid vocabularies with no confidence-calibrated fallback, and left dialogue state tracking, reference resolution, and every generative or executable task entirely out of reach. The eventual limiting factor was not raw accuracy on in-scope inputs but marginal engineering cost: the fifty-first intent cost nearly as much to author and test as the first fifty combined. By the late 1990s, digitised speech corpora and the demonstrated success of probabilistic methods in adjacent fields made a data-driven alternative practically available, shifting the central question from whether statistical models could work to whether the annotation pipelines needed to train them could be built.

Era II: Statistical and Probabilistic NLU (1990s–2000s)

The defining shift of the 1990s and 2000s was the replacement of hand-authored rules with probabilistic models learned from data. This transition was anchored in the statistical speech community, where corpus-trained models had already outperformed hand-engineered rules in large-vocabulary automatic speech recognition [7]. The era also sharpened conversational NLU into two supervised learning problems that persist in production systems today: intent classification, an utterance-level classification task, and slot filling, a token-level sequence-labelling task [8].

Intent classification relied on Naive Bayes and maximum-entropy classifiers. Maximum entropy replaced the generative independence assumptions of Naive Bayes with a discriminative log-linear model supporting arbitrary overlapping features, unigram and bigram indicators, lexical patterns, and

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

gazetteer matches, and it consistently outperformed Naive Bayes because it captured feature interactions and produced well-calibrated posterior probabilities [9]. Slot filling relied first on hidden Markov models and then, more successfully, on conditional random fields. Hidden Markov models condition emission probabilities only on the current tag, which makes "from Boston" and "to Boston" indistinguishable from the emission model alone [10]. Conditional random fields, introduced specifically to correct this and the related label-bias problem found in earlier discriminative variants, model the tag sequence with a single global normalisation term and arbitrary overlapping features, reaching approximately 92 to 95 percent slot F1 on the ATIS benchmark depending on feature richness [11]. These gains came from better hand-designed feature templates, an economy that set up the next era's move away from manual feature engineering entirely.

Statistical NLU made accuracy a function of model class and, critically, of training data. The Penn Treebank institutionalised large-scale human annotation for syntactic information, and the ATIS programme supplied a canonical goal-oriented testbed with audio, transcriptions, and semantic frames [2]. Offline batch pipelines, often built on cluster-computing frameworks descended from MapReduce, orchestrated feature extraction, training, and held-out evaluation [12]. Because conversational systems were frequently speech-first, automatic speech recognition errors compounded directly into downstream NLU: telephony-quality audio routinely produced word error rates in the range of 15 to 30 percent, and system-level accuracy was approximately bounded by the product of per-stage accuracies across the speech-recognition-to-NLU-to-policy cascade.

Statistical NLU also created something closer to a labour market for labelled data. Intent labels, slot tags, and supervised call-routing outcomes became core artefacts of system development, and a growing crowdsourced annotation supply chain emerged to produce them at the scale statistical training demanded. From a systems perspective, the era introduced human-in-the-loop data pipelines — collection, labelling, quality assurance, retraining — that became operationally essential even when model training itself was computationally light [12].

Interactive voice response grew into a multi-billion-dollar industry during this period. Confidence calibration served as the operational contract between the NLU subsystem and the dialogue manager: the classifier did not merely predict an intent; it quantified its own uncertainty, and a hand-authored policy layer consumed that signal to decide whether to proceed or fall back to a human agent. This pattern, a scored NLU prediction feeding a downstream policy layer, survived into later eras even as the model class producing the score changed repeatedly.

Against the scorecard set out in Section 2, the statistical era made intent classification and slot filling production-viable for the first time, but left dialogue state tracking confined to hand-authored state machines and semantic parsing impractical without compositional representations. The era's remaining ceiling was representational rather than architectural: words remained atomic symbols with no learned notion of similarity, so "book" and "reserve" activated entirely separate feature dimensions unless a human engineer built a gazetteer bridging them; multi-turn context was limited to whatever hand-engineered features could be extracted from the immediately preceding turn; and every new domain required a full cycle of data collection and retraining, with no mechanism for transferring learned representations across tasks.

Era III: Neural Sequence Models (2013–2016)

The central discontinuity of this period was the mainstreaming of dense vector semantics. Word2Vec demonstrated that a shallow neural objective trained on unlabelled text learns word vectors that place semantically related terms, "refund" and "money back," near one another in

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

representation space, directly attacking the representational discreteness that had limited the statistical era [13]. Dense embeddings became consequential once paired with sequence models that condition predictions on token order. Recurrent neural networks process tokens sequentially while maintaining a hidden state intended to capture context across an utterance, and early work showed that recurrent architectures could improve slot-filling accuracy on ATIS while exploiting continuous word representations [14]. Plain recurrent networks, however, suffer from vanishing gradients: gradient signals decay exponentially over time steps, making it difficult to learn dependencies beyond roughly ten to twenty tokens [15]. Long short-term memory (LSTM) networks addressed this through gated memory cells that maintain more stable error flow across much longer sequences, though practical natural-language applications still showed degradation beyond roughly fifty to one hundred tokens, a limitation later addressed by attention mechanisms [16].

Sequence-to-sequence models paired LSTM encoders and decoders for end-to-end mapping between input and output sequences, first validated in machine translation and then generalised directly to conversational modelling [17]. Early sequence-to-sequence dialogue models were constrained by a fixed-length bottleneck: compressing an entire input into a single vector discards token-level detail as utterances lengthen. The attention mechanism addressed this by letting the decoder compute a weighted combination over all encoder hidden states at each generation step, replacing "encode everything once" with "retrieve the relevant evidence repeatedly," arguably the single most consequential architectural idea of the decade, since it leads directly to the Transformer [18, 19]. Joint intent-and-slot models built on attention-based recurrent networks improved on prior benchmarks and foreshadowed the unified NLU heads that would follow [20]. Even so, recurrent networks still require step-by-step state updates, so latency grows roughly linearly with sequence length, and training cannot be parallelised across time steps, a constraint that motivated removing recurrence entirely.

This era also began collapsing a pipeline boundary that had defined voice-based NLU since the statistical era: the strict separation between a speech-recognition module and a downstream NLU module. Early end-to-end neural approaches explored replacing several independently trained acoustic and language-modelling components with jointly trained networks mapping speech more directly to text or intent, reducing the error-propagation boundary between recognition and understanding. This trajectory prefigures the multimodal Transformer architectures of later eras, where speech, text, and other modalities are increasingly processed within a single shared encoder rather than a chain of separately optimised modules.

The infrastructure shift was equally significant. GPU programmability, together with optimised deep-learning primitives, made large batches of matrix operations tractable, and industrial frameworks such as TensorFlow formalised dataflow-graph execution across CPUs, GPUs, and clusters, letting teams treat model training as a reusable software discipline with checkpointing and reproducible pipelines [21]. Cloud providers began exposing GPU instances as on-demand resources around 2016, so conversational NLU services could move from on-premises telephony servers toward auto-scaled, pay-per-use GPU infrastructure. Production serving, though, still ran under strict tail-latency budgets, typically under 200 milliseconds end to end for voice assistants, and batching strategies that maximised GPU utilisation for offline workloads conflicted with the need to dispatch requests immediately, so production stacks often operated at batch sizes of one to four, well below GPU-optimal throughput, in order to meet latency service levels.

Dialogue state tracking took its first genuinely learned form during this period: recurrent models

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

mapped speech-recognition output directly to dialogue state without hand-built semantic decoders, reducing the feature-engineering burden relative to statistical-era approaches [22]. Neural multi-label classifiers also made recognising multiple intents in a single utterance, such as "book a flight and reserve a rental car," structurally feasible for the first time. These capabilities, combined with GPU-backed serving, underpinned the mass consumer deployment of intent-based voice assistants during this period; production systems, however, generally paired neural understanding with a still-deterministic dialogue execution layer rather than replacing hand-built policies with learned ones.

Measured against the Section 2 scorecard, the neural era delivered the first workable approaches to stateful multi-turn tasks and made semantic similarity practically useful through dense embeddings, but sequential processing constraints still limited throughput under tight latency budgets, and semantic parsing remained unreliable outside narrow domains. The stateful multi-turn regime was addressable though not yet fully production-robust, and the generative and executable regime remained out of reach, leaving sequential computation as the next bottleneck to resolve.

Era IV: Transformer-Based Pretraining and Transfer Learning (2017–2020)

BERT demonstrated that pretraining a deep Transformer encoder on unlabelled text, then fine-tuning it with a lightweight task head, could match or exceed statistical-era accuracy on intent classification and slot filling using a fraction of the labelled data those methods required [23]. The Transformer's self-attention mechanism, introduced two years earlier, replaced sequential recurrence with parallel computation over the entire sequence, removing the throughput ceiling that had constrained the neural era while also improving long-range dependency modelling [19]. Decoder-only pretraining, explored around the same period, showed that a single generatively pretrained Transformer could be adapted across a range of downstream language tasks with comparatively light task-specific tuning [24, 25].

This paradigm restructured the production NLU pipeline rather than simply improving its accuracy. Feature-extraction stages that statistical-era engineers had hand-built- part-of-speech tagging, n-gram indicators, negation-scope heuristics- were absorbed into the encoder's attention layers as latent representations. What did not shrink, and in some cases grew more complex as accuracy improved, was everything downstream of the encoder: dialogue management and policy, session-state tracking, entity resolution against external knowledge bases, and compliance and redaction logic. Table 1 summarises this redistribution. The net effect was not a shorter pipeline but a restructured one, carrying fewer linguistic feature-engineering stages and heavier orchestration and safety layers.

Table 1: Pipeline Reduction Under Pretrained Transformer Encoders

Internalised Into Encoder	Remained External To The Model
Hand-crafted feature extraction (POS tags, n-gram features) absorbed into encoder attention layers	Dialogue management and policy: flow control, fallback routing, guard rails
Separate embedding lookup tables replaced by dense contextual subword embeddings	Session state and multi-turn tracking: slot carryover, anaphora resolution, dialogue history
Negation and modifier-scope modules (rule-based trigger proximity) captured by multi-head attention	Entity resolution and knowledge-base linking of extracted mentions to canonical records
Sequential dependency chains (linear recurrent processing) replaced by parallel self-attention	Compliance, PII redaction, and business-logic guardrails

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

The infrastructure required to pretrain these encoders was substantially larger than anything the neural era demanded. Custom accelerators such as Google's Tensor Processing Unit delivered large gains in throughput per watt over contemporary GPU and CPU implementations for the matrix-heavy workloads that pretraining requires [26]. Training a single large encoder or generative model no longer fit on one accelerator, which motivated model-parallel training frameworks such as Megatron-LM, which partitions individual Transformer layers across many GPUs [27], and memory-partitioning approaches such as ZeRO, which eliminate redundant optimiser-state replication across data-parallel workers to make larger models trainable on the same hardware budget [28]. On the serving side, distillation techniques such as DistilBERT showed that a smaller student model trained to reproduce a larger teacher's behaviour could retain most of the teacher's accuracy at a fraction of its size and inference cost, making sub-10-millisecond routing latency achievable in cascade architectures that paired a lightweight classifier with a heavier fallback model [29].

Domain-adapted pretraining extended this capability into specialised vocabularies where statistical-era gazetteers had been inadequate: continuing to pretrain or fine-tune an encoder on in-domain text before task-specific adaptation improved entity recognition and slot filling in technical and regulated domains without requiring a return to hand-built lexical resources. The broader implication is that the encoder itself became a reusable infrastructure asset, a shared backbone that could be specialised repeatedly rather than a bespoke artefact rebuilt for every new deployment.

Managed conversational-AI platforms operationalised this shift during 2019 and 2020: a developer defining intents through a visual console was, underneath, triggering a fine-tuning job against a shared pretrained backbone followed by an optimised deployment behind a load-balanced endpoint. By this point most major platforms had converged on Transformer-class backends, and the competitive differentiator shifted from model architecture toward developer experience: visual flow design, one-click deployment, and continuous-integration tooling for dialogue flows.

This convergence introduced a scaling pathology of its own: model sprawl. A multi-domain assistant required one fine-tuned task head per domain, each with its own training pipeline, evaluation suite, and retraining cadence, and multiplying domain count by model version by deployment environment produced a combinatorial matrix of serving surfaces that had to be monitored and rolled back independently, even though the underlying backbone was shared.

Returning to the Section 2 scorecard, the Transformer era resolved the stateless single-turn regime decisively and made the stateful multi-turn regime production-grade for single-domain flows, though the generative and executable regime- semantic parsing, tool invocation, retrieval-grounded generation- remained comparatively nascent. The transition to the next era followed from a specific limitation of fine-tuning itself: BERT-class models had reduced the labelled-data requirement dramatically but still required a distinct fine-tuning cycle for every new domain or task, leaving open the question of whether a single model could generalise across tasks with little or no task-specific training at all.

Era V: The Large Language Model Era (2020–Present)

GPT-3 showed that scaling a decoder-only Transformer to 175 billion parameters produced useful few-shot task performance without gradient-based fine-tuning, using only natural-language instructions and a handful of examples in the prompt [30]. This result depended on an empirical regularity documented shortly before it: cross-entropy loss on held-out data follows an approximately power-law relationship with model

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

size, dataset size, and training compute across several orders of magnitude, which let practitioners plan compute budgets with some confidence that larger, appropriately trained models would perform better [31]. Training GPT-3 required on the order of 3.14 times ten to the twenty-third floating-point operations, several orders of magnitude beyond the pretraining budgets of the Transformer era [30]. Training clusters have since grown to include thousands of accelerators connected by high-bandwidth interconnects, extending the model-parallel and memory-partitioning techniques introduced for BERT-scale training to substantially larger models [32].

Two further developments shaped the era's conversational relevance. Retrieval-augmented generation grounds a generative model's output in retrieved documents by combining a neural retriever with the generator, reducing reliance on parametric memory alone while adding retrieval-index freshness and retriever latency as new operational dependencies [33]. Reinforcement learning from human feedback, used to align GPT-3-derived models into instruction-following assistants, showed that a smaller model tuned on human preference data could be judged more helpful than a much larger untuned model on many tasks, shifting part of the alignment problem from architecture toward a labelled-preference data pipeline reminiscent of the annotation infrastructure built during the statistical era [34]. The subsequent release of large open-weight model families, exemplified by the largest Llama 3 variant reaching 87.3 percent on the MMLU benchmark against 86.5 percent for a leading closed competitor reported around the same period, indicates that pretraining and alignment infrastructure need not remain the exclusive property of a small number of laboratories [35].

Because a single large-model endpoint is far more expensive per request than a fine-tuned classifier, production systems have converged on a tiered architecture rather than routing every utterance through the largest available model. A representative topology separates four layers: ingress and routing, where a lightweight classifier or semantic router triages high-confidence, high-frequency intents to deterministic handlers at low latency and escalates only ambiguous or open-ended queries to the model path; prompt assembly and inference, where a retrieval layer, conversation history, and tool schemas are combined into the model input; execution, where a deterministic workflow engine manages tool calls, transactions, and retries against a separate state store; and safety and observability, spanning content filtering, tracing, and human fallback. This four-layer decomposition reflects the author's own engineering framing of production LLM topologies rather than a documented industry standard. Table II summarises the failure modes distinct to this architecture: misrouting and prompt-injection bypass at the ingress layer, retrieval contamination and hallucinated tool arguments at the inference layer, partial tool failure and stale session memory at the execution layer, and, across all layers, the difficulty of tracing a failed conversation end-to-end when the central component is a stochastic model rather than a deterministic classifier.

Table 2: LLM-Era Failure Modes By Architectural Layer

Layer	Representative Failure Modes	Description
Ingress and Routing	Misrouting; injection bypass	A complex query is dispatched to a deterministic handler that cannot fulfil it; adversarial input evades the pre-generation safety filter.
Assembly and Inference	Retrieval contamination; hallucinated tool arguments	Stale or adversarial documents are injected into the context window; the model generates malformed or fabricated tool-call parameters

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

Layer	Representative Failure Modes	Description
Execution and State	Partial tool failure; stale memory	One of several chained calls fails after others succeed, leaving inconsistent state; the session store reflects a slot value the user has since corrected.
Cross-Cutting	Distributed trace complexity; offline/online metric divergence	Diagnosing a failed conversation requires tracing across router, retrieval, model, and tool; benchmark gains do not reliably predict production task completion

Tool-using large language models decompose compound requests, such as booking a flight, reserving a car, and updating a calendar in one conversational turn, into sequenced actions without hand-authored dialogue policies for each combination. Multi-agent architectures, in which separate planner, researcher, executor, and reviewer roles are assigned to coordinating model instances, represent a current and still-maturing production frontier rather than an established pattern. Much of what appears agentic to end users is, on inspection, orchestration plus tool use plus state management rather than autonomous reasoning by a single model; a deterministic workflow engine still manages execution ordering, transaction boundaries, and retries, while a separate memory subsystem maintains state across turns. This distinction matters architecturally because it clarifies which parts of an agentic system are amenable to conventional software-engineering guarantees and which parts inherit the reliability profile of the underlying model.

This is the article's central claim about the current era: large language models have internalised much of what earlier eras treated as separate NLU subtasks, intent classification, slot filling, dialogue state tracking, and much of dialogue policy, but production deployments have not become simpler as a result. Retrieval, tool validation, state management, safety filtering, and tracing have expanded to fill the space that model-internal reasoning vacated. Where a misclassified intent or a missed slot in earlier eras was local and inspectable, with a clear rollback path, a bad answer from an LLM-based system may originate in retrieval, in the model's own reasoning, in malformed tool-call arguments, or in stale session state, and isolating the cause requires end-to-end tracing across every layer. Operational complexity, in short, has shifted from authoring rules and training classifiers toward monitoring, tracing, and debugging a non-deterministic distributed system whose central component is a probabilistic model.

The Section 2 scorecard registers one further shift here: the LLM era resolves the generative and executable regime that every prior era left out of reach, at the cost of introducing the least inspectable failure surface of the five eras. Whether this trade is favourable depends heavily on deployment context, a question the synthesis below returns to.

Cross-Cutting Synthesis: The Co-Evolution Dynamic

Table 3 summarises the five eras across the four pillars used throughout this article: the scientific innovation that defined each period, the infrastructure it required, the applications it unlocked, and its effect on the surrounding industry. Reading the table across rows shows accuracy gains, application demand, and infrastructure investment moving together rather than in sequence; reading it down columns shows how each pillar's content changed while its function, defining what became feasible, stayed constant.

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

Table 3: The Five Eras Of Conversational NLU Across Four Pillars

Era	Scientific Innovation	Engineering Architecture	Unlocked Applications	Industry Impact
Rule-Based Symbolic (1960s–1980s)	Hand-crafted grammars, frame-and-slot templates, expert-system rule chaining	Single-process rule engines, finite-state dialogue graphs, no retraining pipeline	Scripted chatbots, DTMF/keyword IVR, domain help-desk assistants	Deterministic, auditable systems in regulated telephony; viability limited to narrow domains
Statistical/Probabilistic (1990s–2000s)	MaxEnt classifiers, HMM/CRF slot tagging, n-gram language models	Batch annotation pipelines, offline cluster training, confidence-gated policy layers	Intent-based call routing, containment-optimised IVR	IVR grows into a multi-billion-dollar industry; data annotation becomes its own labour market
Neural Sequence Models (2013–2016)	Word embeddings, LSTM/attention, sequence-to-sequence dialogue modelling	GPU-accelerated training, dataflow frameworks, cloud GPU instances	Context-aware multi-turn assistants, multi-intent recognition	Mass consumer voice-assistant adoption
Transformer/Pretrained Transfer Learning (2017–2020)	Self-attention, encoder pretraining plus fine-tuning, model distillation	TPU and model-parallel training, managed NLU platforms, cascade serving	Low-latency intent/slot classification at reduced data cost, domain-adapted entity recognition	Competition shifts from model architecture to developer experience; model and operational sprawl emerges
Large Language Model Era (2020–present)	Few-shot in-context learning, scaling laws, RLHF alignment, retrieval augmentation	Massive-scale distributed training, tiered inference routing, agentic tool-execution layers.	Zero-shot generative agents, multi-step task completion, retrieval-grounded dialogue	Value chain splits into model builders versus fine-tuners and integrators; open-weight convergence with closed models.

A second cross-cutting pattern is the bottleneck chain summarised in Table 5. Combinatorial brittleness in hand-written rules was resolved by learning from annotated data, which exposed representational discreteness as the next constraint, since words remained atomic symbols regardless of how much data was available. Dense embeddings and recurrent architectures resolved discreteness but introduced vanishing gradients, which gated memory cells, then addressed at the cost of strictly sequential computation. Self-attention removed the sequential constraint and enabled parallel training at scale, which in turn exposed labelled-data scarcity as the binding constraint on any new domain or task. Pretraining, few-shot prompting, and instruction tuning addressed data scarcity, and the current constraint, judging by the failure modes catalogued in Table II, is reliability and governance across a distributed, partly stochastic serving stack rather than any single modelling limitation. Each resolution in this chain required a specific infrastructure investment: batch annotation pipelines, GPU-accelerated training frameworks, model-parallel and memory-partitioning systems, and now retrieval, tool-validation, and tracing layers.

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

Table 5: Sequential Bottleneck Chain Across Eras

Order	Constraint Resolved	Resolving Mechanism	Era	New Constraint Exposed
1	Combinatorial rule brittleness	Learning from annotated corpora (MaxEnt, HMM, CRF)	Statistical	Representational discreteness (atomic word symbols)
2	Representational discreteness	Dense word embeddings	Neural	Vanishing gradients in deep sequential networks
3	Vanishing gradients	Gated memory cells (LSTM)	Neural	Sequential processing constraint (linear latency, limited training parallelism)
4	Sequential processing constraint	Self-attention (Transformer)	Transformer / Pretrained	Labelled-data scarcity per new domain or task
5	Labelled-data scarcity	Pretraining, few-shot prompting, instruction tuning	Large Language Model	Reliability, hallucination, and distributed-system governance

One further observation concerns what has not changed. Core NLU abstractions, intent schemas, dialogue state machines, slot-validation logic, and deterministic fallback paths have survived every architectural transition described above, reappearing inside each new paradigm rather than being discarded by it. The rule-based era's finite-state dialogue manager, the statistical era's confidence-gated policy layer, and the LLM era's deterministic workflow engine perform structurally similar functions with different models behind the score.

A parallel axis of this co-evolution is visible on the application side. The application frontier advanced through qualitative capability stair-steps rather than smooth incremental improvement: scripted chatbots gave way to touch-tone and keyword-spotting IVR, which gave way to intent-based voice assistants, which gave way to context-aware multi-turn assistants, which gave way to generative agents capable of domain-specific task completion. Each step required both a scientific advance and a matching infrastructure capability to arrive together; neither alone was sufficient. Across this arc, successive eras also shifted system behaviour from explicit engineering toward learned representations, which correspondingly changed which components had to be engineered for control, moving from schema validation and dialogue graphs toward routing policies, retrieval grounding, tool-call constraints, and auditability layers. Viewed together with the bottleneck chain in Table IV, this suggests the co-evolution documented here operates on two coupled axes at once: a scientific-bottleneck axis and an application-capability axis, with infrastructure as the mechanism translating progress on one axis into progress on the other.

Several design implications follow from this history for systems architects building the next generation of conversational platforms. Hybrid serving looks like a durable architectural pattern rather than a transitional scaffold: the cost and latency gap between a fine-tuned classifier and a large-model endpoint is structural, not incidental, so production systems benefit from explicit routing layers that direct traffic to the appropriate model tier by query complexity and cost constraint, much as statistical-era systems paired handwritten grammars with early classifiers rather than replacing one with the other outright. Observability and evaluation infrastructure

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

deserve at least as much investment as model selection, since the dominant bottleneck in most enterprise deployments appears to have moved from raw model capability toward the tracing, monitoring, and assurance needed to operate that capability reliably; end-to-end tracing across router, retrieval, inference, tool execution, and safety filter functions as a load-bearing part of the architecture rather than optional instrumentation. Deployment topology is better treated as a variable than a fixed choice, since the same conversational architecture may need to run behind a managed API, on a private GPU cluster, or on an edge device depending on cost, latency, and data-governance constraints; abstractions that treat the model as a pluggable inference service behind a stable interface tend to survive infrastructure shifts better than those that assume one specific serving topology. Safety, guardrail, and tool-execution boundaries warrant the same engineering rigour as the model itself, since these deterministic control surfaces are direct descendants of the slot-validation guards and dialogue-state constraints that enforced correctness in earlier eras, and they become more rather than less important as the model absorbs functions that used to be separate pipeline stages.

Two limits to this account are worth stating plainly. The analysis draws on published research, technical reports, and systems-architecture reasoning rather than on a controlled empirical study conducted for this article, and its quantitative claims are drawn from the cited sources rather than independently re-measured. Latency, cost, and accuracy figures cited for specific systems reflect the conditions reported in their original sources and may not generalise across deployment contexts, hardware generations, or geographic regions.

CONCLUSION

This article has traced conversational NLU from ELIZA's pattern-matching script on a 1960s mainframe to LLM-based agents served from global inference fleets. The argument throughout has been that this trajectory is best read as a co-evolution of scientific accuracy gains and the distributed-systems architecture required to operationalise them, rather than as a sequence of modelling breakthroughs that infrastructure simply followed. At each stage, resolving a scientific bottleneck, combinatorial brittleness, representational discreteness, vanishing gradients, sequential processing limits, or labelled-data scarcity, demanded a new class of infrastructure, from single mainframes to batch clusters to GPU and TPU training fleets to globally distributed inference, and each infrastructure generation unlocked a qualitatively new class of application, from scripted chatbots through IVR automation and voice assistants to generative agents capable of executing multi-step tasks.

The pattern that recurs across six decades, an accuracy breakthrough leading to an infrastructure requirement, leading to a new application category, leading to a rebalancing of operational tradeoffs, appears to hold with reasonable consistency. The survival of core NLU abstractions inside every new architecture is just as persistent, suggesting that engineering discipline, not modelling capability alone, has been a durable determinant of what conversational systems can reliably do.

The claim that large language models represent a simplification of conversational architecture does not hold up under the analysis in Section 7: infrastructure has grown rather than shrunk, with complexity relocating from rule authoring and classifier training toward retrieval grounding, tool validation, and distributed tracing. For practitioners, this suggests that the near-term task is less about replacing earlier stacks than about composing them: deterministic guards, statistical routers, neural classifiers, and generative reasoners operating together under increasingly tight cost, latency, and regulatory constraints. Organisations whose architectures already treat each layer as a separable, observable, and replaceable component appear

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

better positioned to absorb the next shift in this pattern, whatever specific technology drives it.

Two open problems remain understated in the historical account above and deserve explicit mention. Reliability at the frontier has not disappeared as a concern; it has narrowed to the hardest cases, where confidently incorrect output on high-stakes conversational tasks remains a documented failure mode that hybrid architectures and human fallback paths only partly mitigate. The infrastructure trajectory traced here, larger training runs feeding larger inference fleets, also cannot be assumed to continue indefinitely without cost and governance constraints reshaping which architectures remain economically viable at scale. Both problems are, consistent with the argument developed across this article, as much infrastructure and governance questions as they are modelling questions, and addressing them will likely require the same co-evolutionary reasoning applied throughout this account rather than a purely algorithmic fix.

REFERENCES

[1] H. Wang et al., "A Survey of the Evolution of Language Model-Based Dialogue Systems," arXiv preprint arXiv:2311.16789, Nov. 2023. Available: https://arxiv.org/abs/2311.16789

[2] G. Tur, D. Hakkani-Tur, and L. Heck, "What is left to be understood in ATIS?" in Proc. IEEE Spoken Language Technology Workshop (SLT), pp. 19–24, 2010. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2010/12/SLT10.pdf

[3] J. Weizenbaum, "ELIZA - A Computer Program for the Study of Natural Language Communication Between Man and Machine," Communications of the ACM, vol. 9, no. 1, pp. 36–45, Jan. 1966. Available: https://doi.org/10.1145/365153.365168

[4] T. Winograd, "Understanding natural language," Cognitive Psychology, vol. 3, no. 1, pp. 1–191, Jan. 1972. Available: https://doi.org/10.1016/0010-0285(72)90002-3

[5] B. G. Buchanan and E. H. Shortliffe (eds.), Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project. Addison-Wesley, 1984. Available: https://archive.org/details/rulebasedexperts00buch

[6] J. Lighthill, "Artificial Intelligence: a paper symposium (Lighthill Report)," Science Research Council (UK), 1973. Available: https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf

[7] F. Jelinek, Statistical Methods for Speech Recognition. Cambridge, MA: MIT Press, 1997. Available: https://mitpress.mit.edu/9780262546607/statistical-methods-for-speech-recognition/

[8] Y. Wang, L. Deng, and A. Acero, "Spoken language understanding: An introduction to the statistical framework," IEEE Signal Processing Magazine, vol. 22, no. 5, pp. 16–31, Sep. 2005. Available: https://ieeexplore.ieee.org/document/1511821

[9] A. L. Berger, S. A. Della Pietra, and V. J. Della Pietra, "A maximum entropy approach to natural language processing," Computational Linguistics, vol. 22, no. 1, pp. 39–71, 1996. Available: https://aclanthology.org/J96- 1002.pdf

[10] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989. Available: https://doi.org/10.1109/5.18626

[11] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in Proc. ICML, pp. 282–289, 2001. Available: https://dl.acm.org/doi/10.5555/645530.655813

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

[12] J. Dean and S. Ghemawat, "MapReduce: simplified data processing on large clusters," in Proc. OSDI, 2004. Available: https://dl.acm.org /doi/10.1145/1327452.1327492

[13] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in Proc. 1st Int. Conf. Learning Representations (ICLR) Workshop, 2013. Available: https://arxiv.org/abs/1301.3781

[14] G. Mesnil et al., "Using Recurrent Neural Networks for Slot Filling in Spoken Language Understanding," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 23, no. 3, pp. 530–539, 2015. Available: https://doi.org/10.1109/TASLP.2014.2383614

[15] Y. Bengio, P. Simard, and P. Frasconi, "Learning Long-Term Dependencies with Gradient Descent is Difficult," IEEE Trans. Neural Netw., vol. 5, no. 2, pp. 157–166, 1994. Available: https://ieeexplore.ieee.org/document/ 279181

[16] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997. Available: https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext

[17] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 27, pp. 3104–3112, 2014. Available: https://arxiv.org/abs/1409.3215

[18] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015. Available: https://arxiv.org/abs/1409.0473

[19] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008, 2017. Available: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[20] B. Liu and I. Lane, "Attention-Based Recurrent Neural Network Models for Joint Intent Detection and Slot Filling," in Proc. Interspeech, pp. 685–689, 2016. Available: https://arxiv.org/abs/1609.01454

[21] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. OSDI, 2016. Available: https://arxiv.org/abs/1605.08695

[22] M. Henderson, B. Thomson, and S. Young, "Word-Based Dialog State Tracking with Recurrent Neural Networks," in Proc. SIGDIAL, pp. 292–299, 2014. Available: https://aclanthology.org/W14-4340.pdf

[23] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, pp. 4171–4186, 2019. Available: https://arxiv.org/abs/1810.04805

[24] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI Technical Report, 2018. Available: https://cdn.openai.com/research- covers/language- unsupervised/language_understanding_paper.pdf

[25] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI Technical Report, 2019. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[26] N. P. Jouppi et al., "In-datacenter performance analysis of a Tensor Processing Unit," in Proc.

 This work is licensed under a Creative Commons Attribution 4.0 International License.

Article DOI: https://doi.org/10.37284/eajit.9.2.5954

ISCA, 2017. Available: https://arxiv.org/abs/1704.04760

[27] M. Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv preprint arXiv:1909.08053, 2019. Available: https://arxiv.org/abs/1909.08053

[28] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," in Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC), pp. 1–16, 2020. Available: https://arxiv.org/abs/1910.02054

[29] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019. Available: https://arxiv.org/abs/1910.01108

[30] T. B. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020. Available: https://arxiv.org/abs/2005.14165

[31] J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv preprint arXiv:2001.08361, 2020. Available: https://arxiv.org/abs/2001.08361

[32] J. Sevilla et al., "Compute Trends Across Three Eras of Machine Learning," arXiv preprint arXiv:2202.05924, 2022. Available: https://arxiv.org/abs/2202.05924

[33] P. Lewis, E. Perez, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020. Available: https://arxiv.org/abs/2005.11401

[34] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems (NeurIPS), 2022. Available: https://arxiv.org/abs/2203.02155

[35] A. Dubey et al., "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, Jul. 2024. Available: https://arxiv.org/abs/2407.21783

 This work is licensed under a Creative Commons Attribution 4.0 International License.